

Responsive Agents

Dungeon Assistant

Ryan Koenig

ID: 22201444 Email: ryan.koenig@ucdconnect.ie

Generative AI

UCD School of Computer Science

University College Dublin

May 3, 2026

Table of Contents

1	Introduction	3
2	Assistant Design	4
3	Improvements over ChatGPT	5
3.1	Tool Usage	5
4	Results	10
5	Summary	11
6	Acknowledgements	12

List of Figures

3.1	This image shows the train of thought of the agent after being prompted to create a relevant tavern scene, which is connected to a player by name. The agent reads in the campaign notes, creates and image, then takes notes for itself. The simplicity of the request leans into the strengths of the LLM, and thus requires few tool calls to create a good response.	7
3.2	The image generated by Magic Hour. A watermark is included as the agent is connected to the free version of the API. With a more powerful image generation tool the image could be far better.	7
3.3	This image shows the train of thought while the agent creates an encounter. Unlike a typical LLM the agent is able to read the Monster Manual, then runs through a series of balancing passes and information checks. This train of thought performs far better than normal, as seen in the next figures.	8
3.4	The battle map generated by the agent. As with the other images generated, the main limitation is the generation model. Creating a usable battle map is difficult, and this is a valiant attempt. In the future, creating or using an API designed around generating battle maps would serve this function far better.	8
3.5	The stat-block generated by the agent. This enemy represents a high-difficulty city guard outside the tavern, which will come in and break up the fight. The stat-block aligns perfectly with the same enemy in the Monster Manual [5]. This guard is an extremely tough challenge for 2 level 2 players as specified in the prompt, and thus is ;meant to break up the fight. The main fight is the three Bandits in the bar, which pose a moderate threat to the players. The stat-blocks of the Bandits also aligned with the Monster Manual.	9

Chapter 1: Introduction

Large language model assistants have become increasingly capable of supporting complex, domain-specific tasks in recent years. This project explores their application in the context of tabletop role-playing games (TTRPG). For game masters of TTRPGs like Dungeons and Dragons, cognitive load is often the biggest problem limiting the enjoyment of the game master or the players. Whether it be the impossibility of preparing for a session, or the difficulty of coming up with characters, rewards, and encounters in a moment, game masters search for tools that can provide an easier, more fluid experience to playing and running these games.

TTRPGs are a complicated, creative, and thought intensive form of gaming. Game masters often have to utilize a wide breadth of skills. A game master has to make creative decisions like coming up with characters, locations, and story beats on the spot, as well as mathematical analytics and decisions when balancing encounters, items, and other game elements. A game master is a combination of an artist, a writer, a game designer, a mathematician, and an encyclopedia. Alongside this, a good game master must be able to perform these feats quickly, as making your players wait leads to a boring game, even with the most interesting of stories.

This wide spread of skills that must be executed quickly is exactly the problem for which LLM agents and assistants were designed. Modern LLMs have been trained extensively on immense amounts of stories, making them great storytellers. This, combined with a set of tools allowing it to quickly and efficiently create TTRPG materials, makes it the perfect assistant for a game master. Using an assistant like the one created for this project would allow the game master to quickly create story beats, encounters, monsters, items, scenes, battle maps, and more far quicker than they would be able to alone. This allows for faster session preparations, as well as fast during-session adaptations to the actions of the players, making the world feel more alive than ever.

The agent created in this project sets out to make an assistant to a dungeon master from the most popular TTRPG 'Dungeons and Dragons'. This project serves as a baseline agent, with potential to incorporate and update new rule books, as well as a serving as a baseline for the design of similar assistants for other TTRPG games like Cyberpunk or Pathfinder.

Chapter 2: Assistant Design

The assistant presented in this project is a conversational assistant for D&D dungeon masters. To this extent, the agent relies on a stateful loop that processes model requests in order to call and return tool calls and respond to the user with messages once the tool calls are finished. The loop relies on a 'thinking' state, which is determined by whether the agent is calling tools or not. Once the agent no longer calls any tools, its message is relayed to the user.

In order to allow for a conversational approach, the primary agent loop is located inside an input loop. This loop takes input in from the user in order to make using the agent easier, before passing the input to the primary method for the agent to process. The conversational form of this agent requires a list to be used to track the past inputs and outputs. with each input from the user and a tool being appended alongside the outputs of the model. This forms a chain of conversations and outputs that the agent is able to follow.

In addition to the primary loop of the agent, the agent is provided with many tools to enable its functionality. These tools include RAG functionality, API calls to verify information and generate images, deterministic functions, and code interpretation. With these tools and the primary response loop of the agent, the agent is able to use tools to receive, validate, or generate information or images for the user, before reaching a final conclusion that is presented to the user.

Chapter 3: Improvements over ChatGPT

ChatGPT, and other LLMs, are able to provide an extremely useful tool to a lot of game masters but are often limited due to the conversational nature of LLMs, as well as a lack of understanding of the potentially very large rule books for any given TTRPG.' LLMs also often experience response degradation when faced with the immense amount of data and context that can come from a TTRPG campaign, especially if that information is fed through a conversation.

Another problem LLMs face for D&D generations, is that they are unable to provide both an image and a description in the same conversation. This makes it difficult for a DM to create a battle-map for an encounter, while populating the encounter with enemies. Often, an LLM will resort to ASCII art in order to design a battle map, which is unhelpful for a DM, and the players. Encounters, which are the various forms of combat in D&D, also pose a problem for conventional LLMs, as their predictions are often quite poor, both in detail, adherence to the rules, and balance. Experienced dungeon masters often struggle with balancing encounters, and LLMs have little understanding of what makes a fun and balanced encounter.

3.1 Tool Usage

3.1.1 RAG and file retrieval

The most impactful tool provided to this agent is a file retrieval tool, which allows the agent to retrieve three types of files. The first and most important files available to the agent are the three core D&D rule books, the Players Handbook, the Dungeon Master's Guide, and the Monster Manual. This allows the agent to directly scan and reference the rule books when designing new aspects of a campaign or assisting in questions regarding the rules, instead of relying on the memory and training data of the base LLM.

The second type of file available to the agent is simply a notes file. Due to the complexity of TTRPGs, it is often impossible to accurately describe what is happening in a campaign. This notes file allows the agent to record everything it is told about the campaign, whether during preparation or a session, and remember it between conversations, greatly increasing its ability to create more relevant game-play features and advice. This note's file also gives the DM something to look back on, as it is time consuming and difficult to make detailed notes during a session, the notes created by the agent can provide important reminders for what happened, what worked, and what needs to be improved. This type of file retrieval is visible in [Fig: 3.2].

The third type of file are previous notes from the campaign that the DM took. This allows a DM to start using the agent at any point in the campaign. These files are not required for the agent to run, but can help provide context to the agent through any notes the players and DM write separately. For the submission of this project, I have included the notes I made during the session

zero preparations on the overarching story as a demonstration and to help create relevant answers to prompts.

3.1.2 Image Generation

In order to solve the issue of generating images during a conversation, the agent designed in this project has access to Magic Hour's [1] image generation API. This allows the agent to create images for scenes, battle maps, people, and items at request, even while detailing an encounter as shown in the thought process shown in [Fig: 3.3 - 3.5]. Unlike most web LLMs, these images are not fed back to the agent. This helps prevent bloat in the conversation, and thus prevent degradation.

3.1.3 Items

LLMs often struggle to create or remember accurate information about a wide spread of D&D information. Although rule books can help this, they are quite large books that an LLM would struggle to properly remember and refer to. In order to solve this, the agent is provided with an object retrieval function. This function makes a call to Open5e, which is an open-source API that contains the vast majority of information about spells, items, backgrounds, etc. [2]. The agent is then able to use this information about items and spells to better equip enemies, as well as generate rewards. You can see the agent calling this function in [Fig: 3.3].

3.1.4 Encounters and Dice

Encounter balancing is one of the most mathematically demanding tasks a dungeon master faces, and one where conventional LLMs perform particularly poorly. The 2024 rules introduce a specific XP-budget system for balancing encounters [3], where each monster has an XP value and each player character contributes a multiplier based on their level. In order to fix this problem and allow the agent to generate balanced encounters, the agent is provided a deterministic encounter calculator. This is able to take an input of the total XP of the enemies, as well as the player's levels, and returns a difficulty level which can be referenced by the agent to better balance the encounter.

Alongside encounter balancing, the agent is equipped with a dice rolling tool powered by the OpenAI code interpreter [4]. Rather than simulating dice rolls through text generation, where an LLM is prone to bias toward narratively satisfying results, the code interpreter allows the agent to use a pseudo-random number generator. This ensures that any rolls made by the agent during a session are statistically fair.

```

*** Dungeon Assistant ready! Type 'quit' to exit or 'reset' to restart the conversation.

-> It is the start of session 2. The players are heading to a tavern on the outskirts.

- Sending initial message to agent.
- Accessing files
  - Campaign notes
- Calling image generation API
  - Prompt: A cozy workers' tavern on the outskirts of a fantasy town at dusk, warm
- Saving image to disc
- Accessing files
  - Notes
- Taking notes
- Accessing files
  - Notes

Absolutely.

```

Figure 3.1: This image shows the train of thought of the agent after being prompted to create a relevant tavern scene, which is connected to a player by name. The agent reads in the campaign notes, creates and image, then takes notes for itself. The simplicity of the request leans into the strengths of the LLM, and thus requires few tool calls to create a good response.



Figure 3.2: The image generated by Magic Hour. A watermark is included as the agent is connected to the free version of the API. With a more powerful image generation tool the image could be far better.

```

-> Sindri has started a fight in the tavern, and Warren is joining in the fight to protect him.

- Sending initial message to agent.
- Accessing files
  - Statblock
- Accessing files
  - MOM
- Calling image generation API
- Prompt: Top-down fantasy battlemat of a cozy workers' tavern on the outskirts of town and the
- Saving image to disc
- Fetching information from Open5e
- Response: 200, Returning information to agent
- Fetching information from Open5e
- Response: 200, Failed call, notifying agent
- Fetching information from Open5e
- Response: 200, Returning information to agent
- Creating container for dice rolling
- Calculating encounter difficulties.
- Calculated difficulty: Moderate
- Fetching information from Open5e
- Response: 200, Returning information to agent

Here's a **moderately difficult tavern encounter**.

```

Figure 3.3: This image shows the train of thought while the agent creates an encounter. Unlike a typical LLM the agent is able to read the Monster Manual, then runs through a series of balancing passes and information checks. This train of thought performs far better than normal, as seen in the next figures.



Figure 3.4: The battle map generated by the agent. As with the other images generated, the main limitation is the generation model. Creating a usable battle map is difficult, and this is a valiant attempt. In the future, creating or using an API designed around generating battle maps would serve this function far better.

```

## Statblocks

## Warrior Veteran
**AC** 17
**HP** 65
**Speed** 30 ft.
**Initiative Bonus** +3

**STR** 16 (+3)
**DEX** 13 (+1)
**CON** 14 (+2)
**INT** 10 (+0)
**WIS** 11 (+0)
**CHA** 10 (+0)

**Skills** Athletics +5, Perception +2
**Passive Perception** 12
**Languages** Common plus one other

### Actions
**Multiattack.** The warrior makes two Greatsword or Heavy Crossbow attacks.

**Greatsword.**
Melee Attack Roll: +5, reach 5 ft.
Hit: 10 (2d6 + 3) slashing

**Heavy Crossbow.**
Ranged Attack Roll: +3, range 100/400 ft.
Hit: 12 (2d10 + 1) piercing

### Reaction
**Parry.**
Trigger: The warrior is hit by a melee attack roll while holding a weapon.
Response: The warrior adds 2 to AC against that attack, possibly causing it to miss.

```

Figure 3.5: The stat-block generated by the agent. This enemy represents a high-difficulty city guard outside the tavern, which will come in and break up the fight. The stat-block aligns perfectly with the same enemy in the Monster Manual [5]. This guard is an extremely tough challenge for 2 level 2 players as specified in the prompt, and thus is meant to break up the fight. The main fight is the three Bandits in the bar, which pose a moderate threat to the players. The stat-blocks of the Bandits also aligned with the Monster Manual.

Chapter 4: Results

The agent was tested across a variety of dungeon master tasks, ranging from simple questions about rules, to scene descriptions, to full encounter generation. The results show that the agent is capable of assisting a dungeon master in the wide number of tasks they are expected to perform, and produces results that are meaningfully better than an LLM would be able to create through a standard web interface.

For scene generation, the agent is able to produce consistent, campaign relevant descriptions, characters, and plot hooks, as shown in [Fig: 3.2]. Where a prompt is given with no prior history to create a tavern that is connected to a character only referenced in the dungeon master's notes. The agent is able to generate a set of interesting characters with hooks that lead back to the plot, as well as a basic description of the location the dungeon master will be able to expand upon.

In a follow up prompt, the agent is asked to create an encounter inside this tavern, which is where the agent's abilities are best shown. In [Fig: 3.3], the agent's thought processes are visible as it reads information from its sources, generates stat blocks and a battle map, and rolls initiative in a single response. This encounter would normally take a dungeon master a significant amount of time to find, sort through, and create, whereas the agent was able to do so in under a minute, and certainly far exceeds the consistency and abilities of most LLMs.

In spite of this success, there are a few issues with the agent. The most prominent is the quality of the generated images, due to the API given to the agent being a semi-free D&D themed image generator. With better image generators split into separate tools (scene, battle map, character, etc) the agent would likely be able to produce far better images, as the images are quite impressive given the API the agent has access to.

Unfortunately, an integration with the Spotify web API was originally planned, but had to be scrapped due to time constraints. Granting the agent the ability to change playlists and current track would allow the music to adapt to new situations and scenarios the agent is assisting with. In a future version of this project, taking in a Spotify user's information and using the web API to create adaptive music for the session could be a further step in reducing the cognitive load on a dungeon master.

The output of the agent also poses a problem, as a dungeon master would want a quick description and a lot of images that are quick and easy to read. With the current implementation the agent is often limited to printing stat blocks and other descriptions through a long python notebook output. In an application version of the agent, it would be good to immediately present the images, stat blocks, and descriptions generated by the agent in a spread out, easy to understand context.

Chapter 5: Summary

This project set out to create an LLM agent assistant capable of reducing the cognitive load of a Dungeons and Dragons dungeon master. Through myriad tools such as RAG, various APIs, image generation and more, the agent demonstrated a clear capability in helping to create campaign relevant and rule-accurate scenes, characters, encounters and more. These capabilities show great potential in usage as a tool for dungeon masters to assist in active sessions, or session preparations.

The most technically interesting aspect of the project was the agent's primary loop. Where the agent is able to call multiple tools in a single response. Potentially reading the Monster Manual, querying Open5e, calculating encounter difficulty, generating an image, and producing formatted stat-blocks without any intermediate input from the user. This relies on features only provided by the tools in this agent, and provides a speed and accuracy not available through a web based LLM.

The limitations of the project are primarily the APIs used. Higher quality image generation APIs, as well as a better performing ChatGPT API would likely further improve the performance. In an application form of this agent, using better, and more unique, image generation APIs, alongside an improvement on the readability of the outputs of the LLM would greatly increase usability for a dungeon master. A Spotify or music integration would also provide a unique functionality to this project

This project shows that a properly equipped agent focused on helping assist TTRPG game masters would be able to greatly reduce mental load on the game master, increasing speed of sessions and fun for both the game master and their players. While this project is a demonstration of this premise focused through Dungeons and Dragons, future assistants could be tailored for a variety of TTRPG games, or created to be customizable by the user

Chapter 6: Acknowledgements

I would like to thank Professor Tony Veale for his guidance and instruction throughout this module, which provided the foundation for this project.

I would also like to thank my players, whose creativity and support allows me to enjoy and explore D&D, and whose character's information was included in the demonstration materials for this project.

Claude and Gemini were used in the process of implementing the code, mostly through auto-correct or error detection. Claude was also used to help spot errors or issues with the report.

All words in the report are written by me alone, by hand (keyboard).

Bibliography

1. Magic-Hour. *Magic Hour API Documentation* <https://docs.magichour.ai/introduction>. [Online; accessed 29-Apr-2026]. 2026.
2. Open5e. *Open5e API Documentation* <https://open5e.com/>. [Online; accessed 27-Apr-2026]. 2026.
3. Dungeons & Dragons. *Dungeon Master's Guide* (Wizards of the Coast, 2024).
4. OpenAI. *Responses API Documentation* <https://developers.openai.com/api/docs>. [Online; accessed 7-Apr-2026]. 2026.
5. Dungeons & Dragons. *Monster Manual* (Wizards of the Coast, 2024).